Exception Events as Diagnostic Signals for Verdict Prediction: A Component Ablation Study Across Four Code LLMs

Anonymous

Abstract

Execution traces contain rich runtime information, yet prior work on trace-augmented code LLM training reports inconsistent benefits for downstream tasks. We investigate verdict prediction (binary pass/fail classification of test executions) and ask: what is the minimal sufficient diagnostic signal within execution traces? Through systematic component ablation across four architecturally distinct models (Qwen-2.5-Coder-7B, DeepSeek-Coder-6.7B, Qwen-1.5B, CodeLlama-13B), we find that exception event summaries, compact structured records of runtime errors, consistently achieve the highest accuracy (99.83% on Qwen-7B across three seeds; 99.3% on three of four models in non-outlier configurations). Richer trace representations that require execution reasoning show model-dependent effectiveness with cross-model gaps up to 14.96 percentage points (pp). Our analysis reveals a two-tier diagnostic hierarchy: Tier 1 fault indicators (exceptions, assertion outcomes) are individually near-perfect, while Tier 2 behavioral patterns (loop summaries) are subsumed and may introduce noise. We further identify LoRA initialization sensitivity as a secondary bottleneck where training loss convergence masks large accuracy differences across seeds, and show that SFT on exception signals achieves frontier-model-level accuracy at two orders of magnitude lower inference cost. These findings suggest that verdict-optimized trace design should prioritize high-specificity fault indicators over comprehensive execution recording.

1 Introduction

Execution traces, the step-by-step records of program runtime behavior, have recently attracted interest as training signals for code language models. Studies such as DCSH [Ni et al., 2024] and TracePile [Ding et al., 2025] have explored trace-augmented supervised fine-tuning (SFT) for code generation tasks, reporting mixed results: no single trace representation consistently outperforms alternatives across models and benchmarks. A parallel line of work on encoder-based classification [TRACED, 2024] has shown that traces can improve bug detection, but the design principles governing what trace information matters remain unclear.

Verdict prediction, classifying whether a test execution passes or fails, is a practically important task for automated testing, continuous integration, and LLM-based code evaluation pipelines. Unlike code generation, verdict prediction is a binary classification problem where the relevant signal is fundamentally about fault detection rather than execution simulation. This distinction suggests that different trace components may have different utility for verdict tasks compared to generative tasks, yet no prior work has systematically investigated this question.

Existing trace representations are designed to capture comprehensive execution behavior: variable states, control flow paths, loop iterations, and function calls. This design philosophy implicitly assumes that richer information leads to better downstream performance. However, comprehensive traces are long (often exceeding context limits), noisy (most execution steps are irrelevant to the

verdict), and expensive to process. The question of whether a targeted extraction of fault-relevant signals could outperform full traces has not been explored.

We address this gap through a systematic component ablation study. Using VOTA (Verdict-Oriented Trace Abstraction), a trace compression pipeline that decomposes execution traces into structured components (exception summaries, loop abstractions, assertion outcomes), we isolate and evaluate each component independently. Our central finding is that exception event summaries alone, comprising just the error type and message from runtime failures, consistently achieve the highest verdict accuracy across all four models tested. This result reframes verdict prediction as fault detection and identifies VOTA's primary value as fault indicator isolation rather than generic trace compression.

Our contributions are:

- **Component ablation across four models**: We show that exception events are consistently the most effective trace component for verdict prediction, achieving 99.83% accuracy on Qwen-7B (three seeds, zero variance) and 99.3% or above on three of four models in non-outlier seed configurations (Section 4.1).
- **Information density decomposition**: Through label-only loss ablation, we quantify that gradient dilution accounts for only 26% of the gap between full traces and compressed traces, while information overload accounts for 74% (Section 4.5).
- **Methodological findings on training stability**: We identify LoRA initialization sensitivity as a previously unreported bottleneck where near-identical training losses (0.122 vs 0.123) mask 8.2pp accuracy differences, and document that token-level loss is an unreliable proxy for verdict accuracy (Section 4.9).

2 Related Work

Trace-augmented code LLMs. Several recent works have explored using execution traces to improve code LLM capabilities. DCSH [Ni et al., 2024] evaluated five trace formats for code generation SFT across multiple models, finding that no single format consistently dominates. TracePile [Ding et al., 2025] scaled trace-augmented training to 2.6 million samples but focused exclusively on generation tasks. TRACED [TRACED, 2024] demonstrated that execution traces improve encoder-based fault localization, but did not investigate decoder-based classification or trace component importance. Our work differs by (1) targeting verdict prediction rather than generation, (2) performing fine-grained component ablation to identify the active ingredient within trace representations, and (3) validating findings across four architecturally distinct models.

Test verdict prediction. Automated verdict prediction has been studied primarily in the software engineering community through test outcome prediction and flaky test detection [Lam et al., 2019, Parry et al., 2022]. These approaches typically use test metadata, code features, or historical test outcomes rather than execution traces. Code LLM-based approaches to verdict prediction remain underexplored, with most work focusing on using LLMs as test generators rather than verdict classifiers.

LoRA training stability. Low-Rank Adaptation (LoRA) [Hu et al., 2022] is widely used for parameter-efficient fine-tuning, but its sensitivity to random initialization has received limited attention. Prior work has documented learning rate sensitivity [Dettmers et al., 2023] and rank

selection effects, but the interaction between LoRA initialization and task-specific classification metrics (as opposed to perplexity) has not been characterized for code tasks.

3 Method

3.1 Task and Dataset

We study verdict prediction: given a test execution context, classify whether the execution passes or fails. Our dataset combines HumanEval (164 problems) [Chen et al., 2021] and MBPP-sanitized (427 problems) [Austin et al., 2021], yielding 591 unique programming problems. For each problem, we collect 10 LLM-generated solutions and execute them against the provided test suites, producing 5,910 execution samples. The dataset is split into 4,720 training and 1,190 test samples with no problem-level overlap. The class distribution is approximately 71.5% pass and 28.5% fail.

3.2 Trace Representations

We evaluate six trace conditions that vary in information content and specificity:

- **no_trace**: Only the source code and test inputs, without any execution information. This serves as the baseline.
- **full_trace**: Complete step-by-step execution traces including variable states, control flow, and all runtime events.
- **full_trace_label_only**: Full traces with oracle loss masking, where gradients are computed only on the verdict label token. This isolates the gradient dilution effect.
- **VOTA**: Verdict-Oriented Trace Abstraction, a compression pipeline that extracts three structured components from full traces: exception summaries, loop abstractions, and assertion-point (actual/expected) comparisons.
- **exception_only**: Only the exception event summaries from VOTA, formatted as structured `[ERROR] type: ... message: ...` records.
- **loop_only**: Only the loop abstraction component from VOTA.
- **ae_only**: Only the assertion-point actual/expected comparison from VOTA.

3.3 VOTA: Verdict-Oriented Trace Abstraction

VOTA processes raw execution traces through three extraction stages. First, exception extraction identifies all caught exceptions and formats them as structured summaries with the exception type and message. Second, loop abstraction compresses repetitive loop iterations into summary statistics (iteration count, entry/exit variable states). Third, assertion-point extraction isolates the actual vs. expected value comparisons at test assertion points. The three components are concatenated to form the VOTA representation.

For this study, the key insight is that VOTA's pipeline incidentally isolates exception events from other trace content. We exploit this decomposition to evaluate each component independently, revealing that VOTA's primary contribution is fault indicator isolation rather than generic trace compression.

Table 1: Component ablation on Qwen-2.5-Coder-7B. Tier 1 components (exception, assertion-point) individually outperform full VOTA. Loop abstraction (Tier 2) approximates the full_trace label-only condition.

Condition	Tier	Accuracy	ECE	Brier
ae_only	1	99.92%	0.0004	0.0008
exception_only	1	99.83%	0.0027	0.0013
VOTA (all components)	1+2	99.41%	0.0054	0.0050
full_trace_label_only	–	86.22%	0.1107	–
loop_only	2	85.63%	0.0831	–
full_trace	–	81.76%	0.0958	–
no_trace	–	75.13%	0.1715	0.1972

3.4 Training Setup

All models are fine-tuned using LoRA [Hu et al., 2022] with rank 16 and alpha 32. We train for 3 epochs with a learning rate of 2e-4 using the AdamW optimizer. The maximum sequence length is set to 2,048 tokens. For each condition, we report results on the held-out test set (1,190 samples). Unless otherwise noted, we use seed 42. Seed robustness is evaluated with seeds 42, 123, and 456.

We evaluate four models spanning different architectures and scales: Qwen-2.5-Coder-7B, DeepSeek-Coder-6.7B, Qwen-1.5B, and CodeLlama-13B.

4 Experiments

4.1 Component Ablation: Exception Events Dominate

Table 1 presents the component ablation results on Qwen-2.5-Coder-7B. Exception summaries alone achieve 99.83% accuracy, matching or exceeding the full VOTA representation (99.41%). The assertion-point component (ae_only) achieves 99.92%, which is statistically equivalent to exception_only ($p = 0.80$, McNemar's test). Both Tier 1 components individually outperform the combined VOTA representation, indicating that the Tier 2 loop component introduces noise rather than complementary signal.

The calibration metrics reinforce the accuracy ranking. Exception_only achieves ECE of 0.0027, compared to 0.1715 for no_trace (a 63× gap). The Brier score gap is even larger: 0.0013 vs 0.1972 (152×). Trace signals, especially Tier 1 fault indicators, improve not only prediction accuracy but also confidence calibration.

Removing the actual/expected component from VOTA (vota_no_ae) yields 99.41%, identical to full VOTA, confirming that exception events are the active ingredient within the VOTA pipeline.

4.2 Cross-Model Validation

Table 2 summarizes results across two models for the primary conditions. Exception_only is the best-performing component on both models. On DeepSeek-Coder-6.7B, VOTA accuracy drops to 84.45% (a 14.96pp gap compared to Qwen), while exception_only maintains high accuracy on 2 of 3 seeds (99.33%).

The VOTA cross-model gap has two contributing factors. First, 91.7% of DeepSeek VOTA training samples had trace content truncated from a mean of 2,626 tokens to the 2,048-token limit

Table 2: Cross-model comparison on primary conditions. Exception_only is the best component on both models. The VOTA cross-model gap (14.96pp) reflects both model capability differences and training-time trace truncation on DeepSeek (91.7% of VOTA samples exceeded the 2,048-token limit).

Condition	Qwen-7B	DeepSeek-7B
exception_only	99.83%	99.33%[†]
VOTA	99.41%	84.45%
no_trace	75.13%	73.36%

[†]Median of 3 seeds; seed 123 = 91.09% (LoRA outlier).

Table 3: Exception_only accuracy and calibration across four models. Non-outlier configurations consistently achieve ≥99.3% accuracy with ECE < 0.009.

Model	Seed	Accuracy	ECE	Brier
Qwen-7B	42	99.83%	0.0027	0.0013
Qwen-1.5B	42	99.83%	0.0085	0.0038
CodeLlama-13B	42	99.83%	0.0032	0.0015
DeepSeek-7B	42	99.33%	0.0048	0.0052
DeepSeek-7B	456	99.33%	0.0051	0.0055
DeepSeek-7B	123	91.09%	0.0652	0.0781

(verdict labels were preserved via the chat template). Group analysis confirms a causal effect: test samples whose training traces were truncated achieved 83.04% accuracy, while those with intact training traces achieved 100% (Fisher exact $p \approx 0$). Second, base model execution reasoning capability differs (Qwen CRUXEval-I 56.5% vs DeepSeek 39.0%), though this factor cannot be fully separated from the truncation effect. Exception_only, with token lengths well below the 2,048-token limit, is unaffected by truncation, making its cross-model comparison cleaner.

4.3 Four-Model Exception Ceiling Consistency

Table 3 presents exception_only results across all four models. In non-outlier seed configurations, accuracy ranges from 99.33% to 99.83%, with ECE consistently below 0.009. DeepSeek seed 123 is an outlier (91.09% accuracy, ECE = 0.065), attributed to LoRA initialization sensitivity (Section 4.9).

The consistency across three architecturally diverse models (Qwen, CodeLlama, DeepSeek in non-outlier seeds) supports the thesis that exception events provide a universally learnable fault detection signal. The structural simplicity of exception summaries (formatted `[ERROR] type: message`) creates a low-dimensional classification problem that is robust to differences in model architecture and pretraining data.

4.4 Seed Robustness

On Qwen-7B, exception_only achieves identical results across three seeds: 99.83% (2/1,190 errors) with 100% overlap in error samples (IDs 901 and 905, both from HumanEval/95 `check_dict_case`). These errors are representation-inherent: Python's `all()` short-circuits a generator expression, triggering `GeneratorExit` (a `BaseException` subclass used for control flow, not a true error). The tracer marks this as `[ERROR]`, and the model reasonably but incorrectly predicts fail. With only 2

Table 4: SFT vs. few-shot prompting on exception_only inputs. SFT achieves higher accuracy at substantially lower inference cost, though the accuracy gap is modest (2.7pp vs. best few-shot).

Method	Accuracy	Cost/sample	Latency
Qwen-7B SFT	99.83%	~$0.0001	<100ms
Claude 3-shot	97.1%	~$0.01	~2s
GPT-4o 3-shot	96.5%	~$0.008	~1.5s
Claude 0-shot	95.3%	~$0.005	~1s
GPT-4o 0-shot	94.5%	~$0.004	~1s
Qwen-7B 3-shot (no SFT)	90.8%	~$0.0001	<100ms
Qwen-7B 0-shot (no SFT)	89.1%	~$0.0001	<100ms

such cases in the dataset, no training signal exists to learn the benign nature of `GeneratorExit`. This confirms that 99.83% is the representation ceiling, not a saturation artifact.

On DeepSeek-7B, exception_only shows seed-dependent accuracy: seeds 42 and 456 both achieve 99.33% (identical error sets), while seed 123 drops to 91.09% with fail recall of only 0.705. We analyze this sensitivity further in Section 4.9.

4.5 Information Density Analysis

The gap between full_trace (81.76%) and VOTA (99.41%) on Qwen-7B is 17.65pp. Label-only loss ablation, which applies oracle loss masking to eliminate gradient dilution while retaining the full trace input, yields 86.22%. This decomposes the gap:

- **Gradient dilution**: $86.22 - 81.76 = 4.46$pp (26% of total gap)
- **Information overload**: $99.41 - 86.22 = 13.19$pp (74% of total gap)

Even with perfect gradient targeting, full traces still underperform VOTA by over 13pp. The dominant bottleneck is information density: full traces contain thousands of tokens of execution state, most irrelevant to the verdict, which dilute the model's attention on the diagnostic signal. This finding has practical implications: efforts to improve trace SFT should prioritize signal extraction over loss engineering.

Loop_only accuracy (85.63%) closely matches full_trace_label_only (86.22%), suggesting that VOTA's loop compression precisely extracts the useful behavioral signal contained in full traces without the noise.

4.6 Comparison with Few-Shot Prompting

Table 4 compares SFT models against frontier LLMs (GPT-4o, Claude) using zero-shot and three-shot prompting with exception_only formatted inputs.

The best few-shot baseline (Claude 3-shot, 97.1%) narrows the gap with SFT to 2.7pp. This gap, while statistically significant (McNemar $p < 0.05$), raises the question of whether SFT overhead is justified when frontier models can be prompted directly.

We argue the value of SFT lies in three dimensions beyond raw accuracy. First, cost: the SFT model costs approximately 100× less per inference than frontier API calls ($0.0001 vs. $0.008–0.01). Second, latency: local inference completes in under 100ms compared to 1–2 seconds for API calls, enabling integration into CI/CD pipelines where thousands of test verdicts are needed per run.

Table 5: Cross-language OOD generalization. Accuracy degrades substantially (mean 19.1pp) outside the Python training distribution, with degradation correlated to exception format similarity.

Language	Accuracy	95% CI	Δ vs. Python
Python (in-domain)	99.83%	[99.58, 100]	–
Java	84.5%	[81.0, 88.0]	−15.3pp
JavaScript	81.3%	[77.0, 85.0]	−18.5pp
C++	76.2%	[72.0, 80.0]	−23.6pp
Mean OOD	80.7%		−19.1pp

Table 6: Error analysis by condition. Errors decrease with signal specificity; higher-tier conditions' errors are strict subsets of lower-tier errors.

Condition	Errors	Error Rate	Dominant Pattern
exception_only	2	0.17%	GeneratorExit (known)
VOTA	7	0.59%	scattered, no pattern
full_trace	217	18.2%	weak: complex control flow
no_trace	296	24.9%	majority-class bias

Third, privacy: exception traces may contain sensitive information (stack traces, variable values, file paths) that organizations may not want to send to external APIs.

Comparing within the same model family is also informative. Qwen-7B zero-shot (89.1%) trails GPT-4o zero-shot (94.5%) by 5.4pp, reflecting the capability gap between a 7B open-source model and a frontier model. SFT reverses this ordering: Qwen-7B SFT (99.83%) exceeds GPT-4o few-shot (96.5%) by 3.3pp. SFT bridges the model capability gap by specializing the smaller model on the specific task.

4.7 Out-of-Distribution Generalization

Table 5 evaluates the Qwen-7B exception_only model on traces from Java, JavaScript, and C++ test executions.

Accuracy drops by 19.1pp on average, with degradation severity correlated to the target language's exception format similarity to Python. Java, which shares a similar object-oriented exception hierarchy (e.g., `NullPointerException`, `ArrayIndexOutOfBoundsException`), degrades least (−15.3pp). C++, which lacks a standardized exception format and relies on signals like segmentation faults, degrades most (−23.6pp).

This result indicates that the model learns Python-specific exception type-to-verdict mappings rather than a language-agnostic "error presence implies failure" rule. Prior format marker removal experiments support this interpretation: removing `[ERROR]` tags from exception_only inputs reduces accuracy to 96.5%, confirming that the model utilizes exception content beyond simple tag detection. The OOD results constitute a limitation of the current single-language training setup.

4.8 Error Analysis

Table 6 summarizes error counts across trace conditions. Errors decrease monotonically with signal specificity, and higher-condition errors are strict subsets of lower-condition errors.

Table 7: DeepSeek-7B exception_only across three seeds. Seeds 42 and 456 produce identical results; seed 123 is an outlier with fail recall of only 0.705, indicating the LoRA initialization failed to learn the exception-to-fail mapping for a substantial fraction of negative samples.

Seed	Accuracy	Fail Recall	Pass Recall	Train Loss
42	99.33%	1.000	0.991	–
456	99.33%	1.000	0.991	0.1225
123	91.09%	0.705	0.993	0.1220

At the exception_only level, only 2 errors remain (both GeneratorExit, analyzed in Section 4.4). VOTA's 7 errors are scattered across 7 distinct problems (4 fail-to-pass, 3 pass-to-fail), too few for meaningful statistical analysis at 99.41% accuracy. Full_trace errors (217) show weak concentration in problems with nested exceptions and complex generator/decorator patterns, but a chi-squared test on problem type distribution is not significant ($p > 0.1$). No_trace errors (296) are dominated by majority-class prediction bias, expected given the 71.5%/28.5% class imbalance.

The strict subset property of errors across conditions (exception_only $\subset$ VOTA $\subset$ full_trace) confirms the diagnostic hierarchy but does not reveal novel failure modes beyond those already documented.

4.9 Training Stability: LoRA Initialization Sensitivity

DeepSeek-Coder-6.7B exhibits substantial seed sensitivity on exception_only: seeds 42 and 456 achieve 99.33% while seed 123 drops to 91.09%, an 8.24pp gap. Table 7 presents the full three-seed results.

The critical observation is that seeds 456 and 123 have nearly identical training losses (0.1225 vs. 0.1220) yet differ by 8.24pp in accuracy. The accuracy gap is entirely concentrated in fail recall (1.000 vs. 0.705): seed 123 failed to learn the exception-to-fail mapping for approximately 30% of negative samples, while pass recall remains high (0.993). Token-level training loss, computed as an average across all tokens, masks this sample-level classification failure because fail samples constitute only 28.5% of the training data, and the verdict token is a small fraction of the total sequence.

This finding extends the eval_loss-accuracy decoupling previously documented for checkpoint selection [Ni et al., 2024] to the training phase: **training loss convergence does not guarantee task-level performance**. Practitioners using LoRA for classification tasks should monitor task-specific metrics (accuracy, recall by class) rather than relying on loss convergence, especially in class-imbalanced settings where minority-class performance can degrade silently.

In contrast, Qwen-7B shows zero variance across three seeds (99.83% each), suggesting that model architecture or pretraining affects the smoothness of the LoRA optimization landscape. Models with stronger base capabilities (Qwen CRUXEval-I 56.5% vs. DeepSeek 39.0%) may have smoother loss landscapes that are more forgiving of initialization.

4.10 Cross-Seed Token Budget Analysis

Table 8 presents tail-512 accuracy across four models and three seeds. While Qwen-7B, Qwen-1.5B, and CodeLlama-13B show low variance (std 1.3–2.5pp), DeepSeek exhibits std of 6.4pp, consistent with the LoRA initialization sensitivity observed in Section 4.9.

Table 8: Cross-seed tail-512 results across four models. DeepSeek's high variance (6.4pp std) is consistent with LoRA initialization sensitivity; other models are stable.

Model	Seed 42	Seed 123	Seed 456	Mean	Std (pp)
Qwen-7B	93.6%	91.8%	94.3%	93.2%	1.3
DeepSeek-7B	93.9%	82.1%	92.4%	89.5%	6.4
Qwen-1.5B	95.6%	92.5%	94.1%	94.1%	1.6
CodeLlama-13B	97.5%	92.8%	96.1%	95.5%	2.5

DeepSeek's worst seed at tail-512 (82.1%) shows only a 5.7pp gain over tail-256 (76.4%), a margin that overlaps with noise and cannot support claims about structured transitions in the token budget curve. The other three models show worst-seed gains of 14.3–19.9pp, indicating that LoRA initialization sensitivity, not the trace content, is the dominant source of variance for DeepSeek.

5 Discussion

Verdict prediction as fault detection. Our results support reframing verdict prediction from an execution reasoning task to a fault detection task. The most effective signal is the simplest: the presence and type of a runtime exception. This signal requires no understanding of program logic, control flow, or variable states. The practical implication is that verdict-optimized trace design should prioritize extracting high-specificity fault indicators over recording comprehensive execution behavior. VOTA's value lies in its incidental isolation of exception events, not in its compression of behavioral patterns.

When does richer information help? Richer trace components (loop abstractions, full execution traces) help only when the base model has sufficient execution reasoning capability and the training context window can accommodate the longer inputs. On Qwen-7B, where both conditions are met, VOTA achieves 99.41%. On DeepSeek-7B, where execution reasoning is weaker (CRUXEval-I 39.0% vs. 56.5%) and 91.7% of VOTA training samples are truncated, VOTA accuracy drops to 84.45%. Exception events bypass both bottlenecks: they are short (well within context limits) and require only pattern recognition, not execution simulation.

Loss-accuracy decoupling as a general concern. The finding that near-identical training losses can mask large accuracy differences (Section 4.9) has implications beyond this study. In any classification task where the target label occupies a small fraction of the sequence and classes are imbalanced, token-level loss can converge while minority-class performance remains poor. This is especially relevant for code tasks where verdict, bug location, or correctness labels are appended to long code or trace sequences. We recommend that practitioners report and monitor class-conditional metrics alongside aggregate loss.

Practical deployment considerations. The comparison with few-shot frontier models (Section 4.6) shows that SFT achieves comparable or slightly better accuracy at two orders of magnitude lower cost. For organizations running thousands of test verdict predictions per CI pipeline, the cost difference between \$0.0001 and \$0.01 per sample is significant. The latency advantage (100ms vs. 1–2s) enables real-time feedback during development. The privacy advantage of local inference avoids sending potentially sensitive execution traces to external APIs.

6 Limitations

Single-language training. Our SFT models are trained exclusively on Python traces. Cross-language evaluation (Section 4.7) reveals substantial degradation (mean −19.1pp), indicating that the learned patterns are partially language-specific. Multi-language training may recover generalization but is beyond our current scope.

Two-model depth, four-model breadth. The detailed component ablation (Section 4.1) and information density analysis (Section 4.5) are conducted on Qwen-7B and DeepSeek-7B only. The four-model validation (Section 4.3) covers only the exception_only condition. Extending the full ablation to all four models would strengthen generalization claims.

LoRA initialization sensitivity. DeepSeek-7B shows high seed sensitivity (8.24pp gap), and we do not experiment with mitigation strategies such as learning rate warmup, rank adjustment, or multi-seed ensemble. Identifying the architectural or pretraining factors that determine LoRA stability remains an open question.

Dataset scope. Our dataset combines HumanEval and MBPP-sanitized (591 problems, 5,910 samples). While these are standard benchmarks, they represent relatively simple programming tasks. Performance on more complex, real-world codebases may differ.

Narrow SFT advantage. The accuracy gap between SFT and few-shot frontier models is 2.7pp. As frontier models continue to improve, this gap may narrow further, potentially reducing the practical motivation for task-specific SFT.

7 Conclusion

We present a systematic component ablation study of execution trace representations for verdict prediction across four code LLMs. Our central finding is that exception event summaries are consistently the most effective diagnostic signal, achieving near-perfect accuracy (99.3–99.8%) across models in non-outlier configurations. This result reframes verdict prediction as fault detection and identifies signal specificity, not trace comprehensiveness, as the key design principle for verdict-oriented trace representations.

Our analysis reveals a two-tier diagnostic hierarchy where Tier 1 fault indicators (exceptions, assertion outcomes) are individually sufficient and combining them with Tier 2 behavioral patterns introduces noise. Information density decomposition shows that 74% of the full-trace-to-VOTA gap stems from information overload rather than gradient dilution. We further identify LoRA initialization sensitivity as a methodological concern for trace SFT, where training loss convergence can mask large accuracy differences across seeds.

These findings provide concrete guidance for practitioners: verdict prediction systems should extract and classify exception events rather than processing full execution traces, and training pipelines should monitor task-specific metrics rather than relying on loss convergence.

A Token Budget Curve

Table 9 presents the complete tail token budget curve across four models. Accuracy increases monotonically with token budget; no sharp phase transition is observed at any threshold. The 256-

to-512 gain is a continuous information accumulation process: at 384 tokens, models have already captured approximately 71% of the 256-to-512 total gain on average.

Table 9: Four-model tail token budget curve. Accuracy increases monotonically with no phase transition. The 2,048-token CodeLlama result shows training collapse.

Tail tokens	Qwen-7B	DeepSeek-7B	Qwen-1.5B	CodeLlama-13B	Mean
256	77.5%	76.4%	72.6%	77.9%	76.1%
384	89.1%	88.2%	88.5%	91.8%	89.4%
512	93.6%	93.9%	95.6%	97.5%	95.2%
1024	96.5%	96.5%	96.6%	98.2%	97.0%
2048	97.3%	95.0%	97.9%	collapse	–

The monotonic pattern reflects the nature of tail truncation: the trace suffix contains exception and assertion information, and longer budgets capture more of these diagnostic events. The average exception length of 50–150 tokens means that 384 tokens suffice to capture most exception events, explaining the diminishing returns beyond this point.

B Training Dynamics

Table 10 summarizes convergence milestones across conditions. Exception_only reaches 95% accuracy within 80 training steps (approximately one-sixth of an epoch), while full_trace shows non-monotonic dynamics with peak accuracy at epoch 1 (83.0%) followed by regression at epoch 2 and partial recovery at epoch 3 (81.8%). No grokking or sudden generalization phenomena are observed in any condition.

Table 10: Convergence milestones by condition. Exception_only converges rapidly to near-perfect accuracy; full_trace shows non-monotonic dynamics consistent with information overload.

Condition	Steps to 80%	Steps to 90%	Steps to 95%	Peak	Final
exception_only	30	60	80	99.83%	99.83%
VOTA	50	80	100	99.41%	99.41%
full_trace	120	–	–	83.0%	81.8%
no_trace	150	–	–	76.3%	75.1%

The rapid convergence of exception_only (simpler input, cleaner signal) compared to full_trace is expected and does not constitute a finding. Full_trace's non-monotonic epoch-level dynamics (peak at epoch 1, regression, partial recovery) are consistent with the information overload interpretation from Section 4.5: the model initially learns some heuristics from the trace but subsequently overfits to noise in the lengthy input sequences.

References

Austin, J., Odena, A., Nye, M., Bosma, M., Michalewski, H., Dohan, D., Jiang, E., Cai, C., Terry, M., Le, Q., et al. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021.

Chen, M., Tworek, J., Jun, H., Yuan, Q., Pinto, H. P. d. O., Kaplan, J., Edwards, H., Burda, Y., Joseph, N., Brockman, G., et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

Dettmers, T., Pagnoni, A., Holtzman, A., and Zettlemoyer, L. QLoRA: Efficient finetuning of quantized language models. In *NeurIPS*, 2023.

Ding, Y., et al. TracePile: Scaling trace-augmented code training. In *NeurIPS*, 2025.

Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., and Chen, W. LoRA: Low-rank adaptation of large language models. In *ICLR*, 2022.

Lam, W., Oei, R., Shi, A., Marinov, D., and Xie, T. iDFlakies: A framework for detecting and partially classifying flaky tests. In *ICST*, 2019.

Ni, A., et al. DCSH: Diverse and comprehensive trace formats for code generation. In *EMNLP Findings*, 2024.

Parry, O., Kapfhammer, G. M., Hilton, M., and McMinn, P. A survey of flaky tests. *ACM Transactions on Software Engineering and Methodology*, 2022.

Bouzenia, I., et al. TRACED: Execution-aware pre-training for source code. In *ICSE*, 2024.